

PRGR Launch Monitor — Complete Project Briefing

A single, self-contained overview of the PRGR DIY golf launch monitor: what it is, the full hardware stack with specs, how everything connects, power, the development plan, and where the project stands.

Last updated: 2026-07-01

Contents

1. What it is
2. Goals
3. Hardware — full component specs (compute & display, camera + lens, radar array, impact trigger, power, BOM)
4. How it all connects
5. Software stack
6. Development sequence & current status
7. Key open question — spin detection
8. The short version

1. What it is

A **DIY golf launch monitor** — a device that measures what happens to a golf ball and club at the moment of impact and for the first few feet of flight, then calculates the full shot: **carry distance, total distance, ball speed, club speed, launch angle, spin, and smash factor**.

Everything runs on a **Raspberry Pi 5** with a touchscreen. A high-speed camera watches the ball for spin, and three Doppler radars measure speed and angles. All processing happens on-device — no phone app, no cloud service, no subscription. You hit a ball and the numbers appear on the screen.

The architecture is modeled after the commercial **Rapsodo MLM2 Pro** (a camera for spin + Doppler radar for speed/angle, fused together), assembled from off-the-shelf sensors and open hardware at a fraction of the retail cost.

2. Goals

Produce **accurate, repeatable shot data** for a golfer practicing indoors or at a range, from a device that:

1. **Detects the moment of impact** (camera + radar + optional sound trigger)
2. **Measures ball speed and club speed** (Doppler radar)
3. **Measures launch angle and club path** (twin radars at different orientations)
4. **Detects ball spin** (high-FPS camera watching the ball leave the tee)
5. **Fuses all sensor inputs** into a single confident shot reading
6. **Calculates carry and total distance** via a ballistics model

7. **Presents it all** on a fast, touch-first 800×480 screen

Design principles: on-device only, sensor fusion over any single source, touch-first UI, and a Development Mode so every screen works with simulated data (no hardware needed) for UI work and demos.

3. Hardware — full component specs

Compute & display

Component	Spec	Role
Raspberry Pi 5 (8 GB)	Quad-core Arm Cortex-A76 @ 2.4 GHz, aarch64	All processing: camera pipeline, vision, sensor fusion, UI
Waveshare 5" DSI LCD	800 × 480 IPS, DSI @ 60 Hz, 5-point capacitive touch, 6H tempered glass, ~1.2 W	The entire touchscreen UI
microSD / storage	Raspberry Pi OS Bookworm (64-bit)	OS, app binary, shot history, calibration

The display's native **800 × 480** matches the UI target exactly — no scaling, every pixel maps 1:1. It connects over the **DSI ribbon** (not HDMI) and is driver-free.

 **Pi 5 cable gotcha:** the Raspberry Pi 5 uses a smaller 22-pin DSI connector, so this panel needs the correct adapter cable for the Pi 5 (Waveshare Pi5-Display-Cable-200mm or the official Pi 5 DSI cable). The cable in the box is for older 15-pin Pis.

Links: Raspberry Pi 5 · Waveshare 5" DSI LCD

Impact camera + lens

The single most important sensor for spin detection — it watches the ball for the first few feet of departure at very high frame rates.

SENSOR (OV9281)

Attribute	Spec
Sensor	OmniVision OV9281 — global shutter, monochrome, 1 MP
Native resolution	1280 × 800
Pixel pitch	3.0 μm
Sensor size	3.84 mm × 2.40 mm
Interface	CSI (MIPI) via libcamera / rpicalm-vid
Orientation	Rotated 90° clockwise (portrait) to maximize vertical coverage of the departure path
Preview mode	640 × 480 sensor → 480 × 640 portrait @ 180 FPS (after 90° CW rotation)
Capture mode	640 × 400 sensor → 400 × 640 portrait @ 240 FPS (after 90° CW rotation)

LENS (8 MM F1.2 M12)

Attribute	Value
Focal length	8 mm
Aperture	F1.2 (fixed iris)
Mount	M12
Field of view (D × H × V)	50° × 41° × 31°
Minimum object distance	0.2 m (20 cm)
Back focal length (BFL)	6.58 mm
Construction	7 elements in 6 groups, aluminum-alloy barrel
IR correction (day/night)	Yes
Focus / Zoom	Manual focus, fixed zoom
Dimensions	Ø16 mm × 30.2 mm
Operating temp	−20 °C to +80 °C

Why global shutter? A global-shutter sensor exposes every pixel at the same instant. A rolling shutter smears and skews a fast-moving ball. For measuring spin and speed at impact, global shutter is non-negotiable.

Why monochrome + IR + F1.2? Mono sensors collect light on every pixel — critical for the short (~100–300 μs) exposures that freeze motion. The F1.2 aperture and IR-corrected lens make those microsecond exposures possible under indoor lighting.

Distance from ball — TBD. ~5 ft is a *starting hypothesis*, **not yet validated** (as of 2026-07-01). The real distance will be derived from the radars' accurate range plus the ball's required pixel size (see §6). Theoretical ball size with the 8 mm lens:

Distance	Ball pixel diameter (theoretical)
5 ft	~75 px
6 ft	~62 px
7 ft	~53 px
8 ft	~47 px

Links: Arducam OV9281 for Pi · libcamera docs

Radar array

Three Doppler radars provide speed and angle data, cross-checking the camera. Drivers are vendored from the open-source **OpenFlight** project (Python), then bridged to the C++ app.

Module	Frequency	Interface	Role	Manuals
OPS243-A	24.125 GHz	USB / UART / RS-232	Ball speed, club speed, spin backup (I/Q)	Product · API (AN-010) · User Manual
K-LD7 #1 (vertical)	24.05–24.25 GHz	UART (3.3 V)	Launch angle (vertical)	Product · Datasheet
K-LD7 #2 (horizontal)	24.05–24.25 GHz	UART (3.3 V)	Club path / aim (horizontal)	Product · Datasheet

OPS243-A — details: K-band Doppler radar, 24.125 GHz, 1–100 m detection range, speed up to 348 mph, inbound/outbound direction, ~20° × 24° beam. Outputs raw **I/Q** samples (the reason it was chosen — needed for spin/impact buffering). Default UART 19,200 baud. –40 to +85 °C.

K-LD7 (×2) — details: 24 GHz fully-digital radar transceiver measuring speed, direction, distance, and **angle**. 3.2–5.5 V supply, ~20–60 mA, 3×4 patch antenna, on-board DSP with a target list and tracking filter. Run at **3 Mbaud** in this project. One oriented for vertical angle (launch angle), one for horizontal (club path).

⚠ Radar hardware notes:

- Buy the **OPS243-A**, *not* the OPS243-A-W (WiFi) — the WiFi version's baud rate is too slow to stream I/Q data.
- The K-LD7 modules require **3.3 V** FTDI USB-serial adapters. A **5 V adapter will damage** the module. Do not exceed 5.5 V supply.

Impact trigger (under evaluation — optional)

Component	Spec	Role
SparkFun SEN-14262 Sound Detector	Analog + gate + envelope outputs; preamp gain set by R17	Detects the <i>sound</i> of impact to trigger the radar's rolling I/Q buffer capture

🤔 **Still deciding whether this is needed.** OpenFlight uses a sound trigger to mark the *exact instant* of impact so the right slice of the OPS243-A rolling I/Q buffer can be captured. But PRGR already has **two other impact markers**: the **radar** (a sudden speed-threshold crossing) and the **camera** (`IMPACT_DETECTED` from the ball-zone state machine). The sound trigger's only real advantage is sub-frame **timing precision**. **Current plan**: start with radar/camera triggering and treat the sound detector as **optional**, adding it later only if the I/Q capture window proves hard to hit without it. Acoustic triggering in a bay has downsides (false triggers from mat noise / reflections, plus the R17 gain tuning) that aren't worth it unless the precision proves necessary.

🔧 **R17 mod (if used)**: R17 is an unpopulated resistor footprint that sets the preamp gain. Populating it (~33–47 kΩ) in parallel with R3 lowers the gain so the detector isn't permanently saturated at 3.3 V — otherwise the GATE output latches high and never registers a distinct impact.

Links: Product · Hookup Guide

Power

The Raspberry Pi 5 is power-sensitive. It needs a **5 V / 5 A (25 W)** USB-C source that performs a proper Power Delivery (PD) handshake. **Without that handshake the Pi 5 caps total USB current to 600 mA**, which starves the three-radar array and causes capture failures. A genuine 5 A supply unlocks the Pi's **~1.6 A USB budget** — the amount needed to run the OPS243-A + 2× K-LD7 simultaneously. (*This limit is well-documented by the OpenFlight project, whose Pi 5 + triple-radar + touchscreen stack is the same as ours.*)

Three ways to power the build:

Scenario	Source	Notes
Bench / indoor	Official Raspberry Pi 27 W USB-C PD (5 V / 5 A)	Reference standard. Guarantees the PD handshake.
Mobile / range (simple)	5 V / 5 A PD-compliant USB-C power bank (25 W)	Must advertise 5 V / 5 A , not just "25 W" at higher voltages.
Integrated tower (our build)	TalentCell PB120B1 (12 V, 142 Wh) → 12 V-to-5 V/5 A USB-C buck converter	Self-contained battery for a tower enclosure.

The tower / TalentCell approach. For a self-contained tower we use a **TalentCell PB120B1** — a 12 V lithium pack, **38,400 mAh / 142 Wh**, 3s4p 18650 cells, with a **12 V / 6 A DC** output and a **5 V / 2.4 A** USB output. Wiring it correctly matters:

- ⚠️ **Do NOT power the Pi 5 from the TalentCell's own 5 V USB port** — it's only **2.4 A**, below the Pi 5's requirement, and it does not do a PD handshake.

- Instead feed the **12 V DC output** (up to 72 W available) into a **12 V → 5 V / 5 A USB-C buck converter** (a 25 W, 5 A-rated module), and run that into the Pi.
- A plain buck converter is **not PD-compliant**, so add `usb_max_current_enable=1` to `/boot/firmware/config.txt` to unlock the full USB budget for the radars without a PD negotiation.
- Use **5 A-rated USB-C pigtails / ≥ 20 AWG wire** between the buck and the Pi.

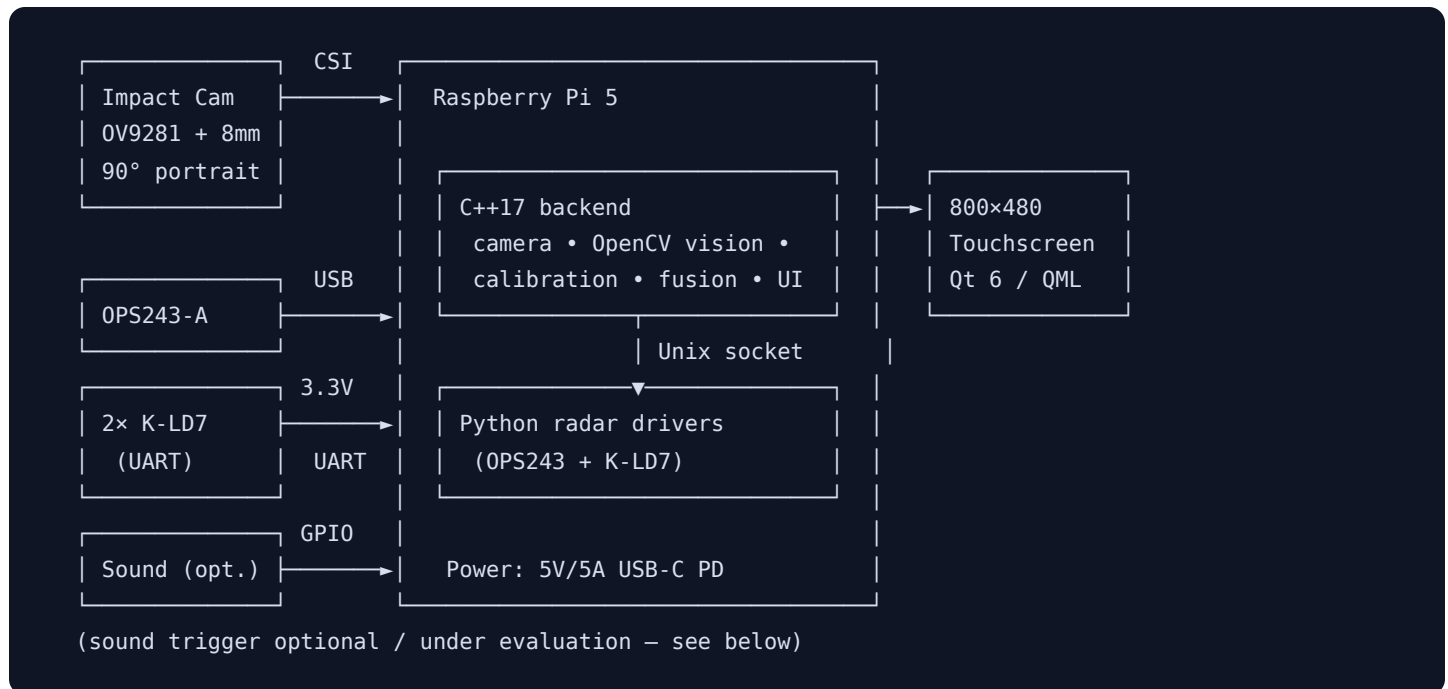
Runtime: at ~15–25 W typical draw (Pi + 3 radars + 5" screen), the 142 Wh pack gives roughly **5–7 hours** of range time (buck efficiency ~90 %).

Bill of materials

Item	Approx. cost
Raspberry Pi 5 (8 GB)	~\$80
Waveshare 5" DSI LCD + Pi 5 DSI cable	~\$40
OV9281 camera + 8 mm F1.2 M12 lens	~\$40
OPS243-A Doppler radar	~\$249
K-LD7 radar ×2	~\$120
3.3 V FTDI USB-serial adapters ×2	~\$20
SparkFun SEN-14262 sound detector (+ 47 kΩ resistor) — <i>optional, under evaluation</i>	~\$13
TalentCell PB120B1 + 12 V → 5 V/5 A buck converter	~\$110
Wiring, connectors, 3D-printed enclosure	~\$30
Approx. total	~\$700

The radar add-on alone (OPS243-A + 2× K-LD7 + adapters + trigger + wiring) is ~\$412 if you already have the Pi, screen, and camera.

4. How it all connects



- **C++17 backend** does the heavy lifting: camera control, ball detection (OpenCV 4), calibration, sensor fusion, and the UI logic.
- **Python** runs the radar drivers (vendored from OpenFlight) as a **child process**. C++ and Python communicate over a **Unix domain socket** (`/tmp/prgr_radar.sock`) using newline-delimited JSON — message types: heartbeat, status, shot_data, error. Python sends a heartbeat every 2 s; C++ flags the radar "offline" after 3 missed beats.
- **Qt 6 / QML** is the touchscreen front-end — a fast, finger-first UI at exactly 800×480, reading state from the C++ managers via `Q_PROPERTY` bindings.

5. Software stack

Layer	Technology	Purpose
Backend	C++17	Camera pipeline, ball detection, sensor fusion, UI managers
Frontend	Qt 6 / QML	Touch-first 800×480 UI
Vision	OpenCV 4	Ball detection (Hough, blob, contour, MOG2), Kalman tracking, calibration
Radar drivers	Python 3.10+	OPS243-A & K-LD7 serial drivers (vendored from OpenFlight, AGPL-3.0)
Camera capture	libcamera / rpicam-vid	High-FPS YUV420 capture, Y-channel extraction for mono processing
Build	CMake	Pi is the primary target; Windows desktop staging build supported (Dev Mode only)

Ball detection uses multi-method scoring: Hough circle transform, blob detection, contour finding, and MOG2 background subtraction, with the highest-confidence result chosen. A Kalman filter and a ball-zone state machine (NO_BALL → STABLE → READY → IMPACT_DETECTED) handle in-frame tracking and the impact trigger.

Development Mode swaps real hardware for simulated data so every screen works with no camera or radar attached — this is how UI work and demos happen on a laptop.

6. Development sequence & current status

The physical geometry of the build is **not** being guessed up front — it's established empirically, **radar-first**:

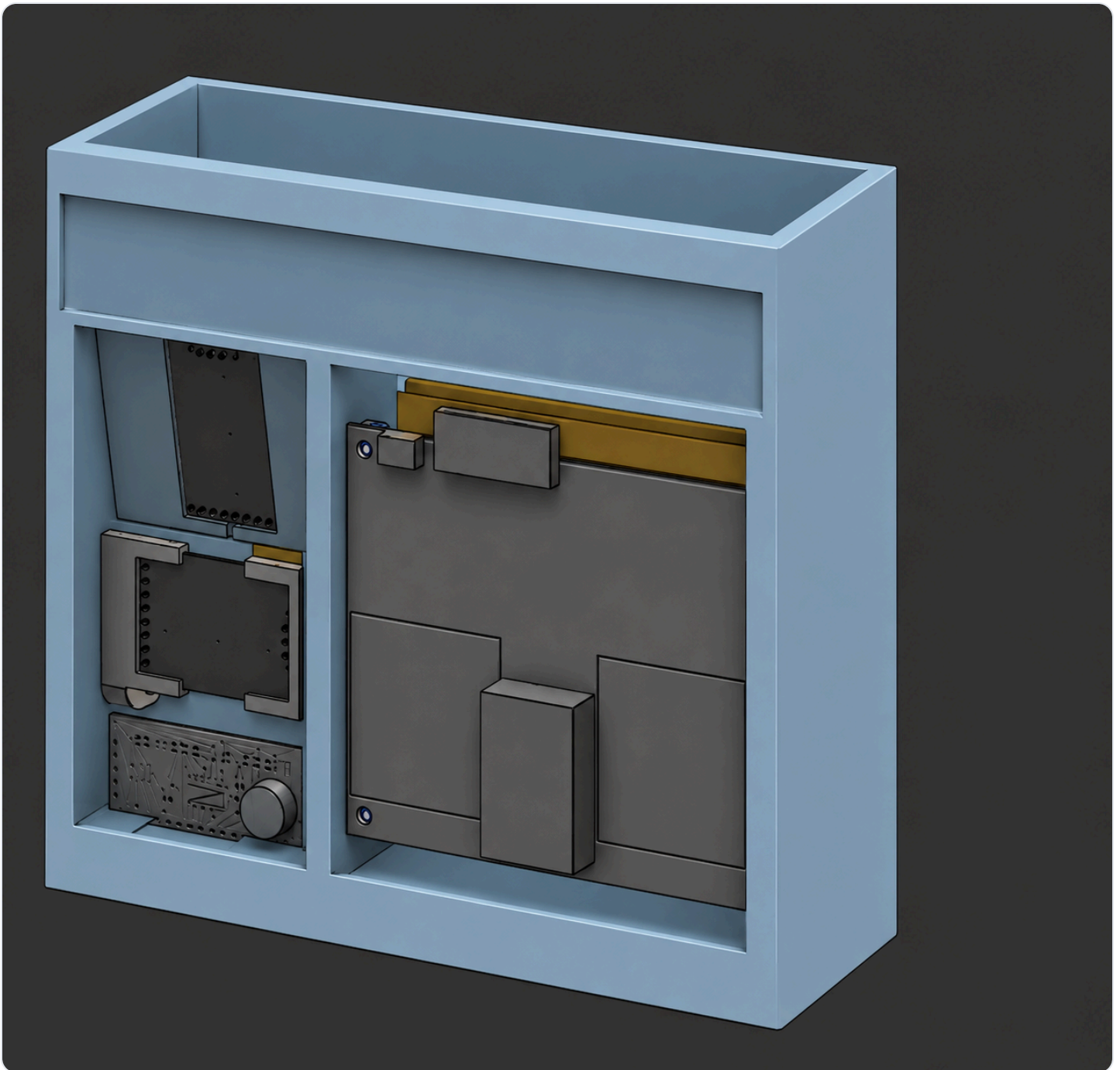
1. **Get the radars working** — OPS243-A + 2× K-LD7 (OpenFlight-vendored Python drivers). Prove we can read ball speed, club speed, launch angle, and club path. (*current priority*)
2. **Establish the radars' accurate range** — the **maximum distance** at which the radars return trustworthy data. This becomes the anchor constraint for everything else.
3. **Incorporate the impact camera** — only once radar range is known, work out how far the ball must be so the ball's **pixel diameter** is large enough for spin detection, the FOV covers the departure path, and the distance is compatible with the radar's range.
4. **Calibrate and measure** — with real, fixed geometry, run calibration and lock in the true distances.

Status (as of 2026-07-01):

Done	In progress / planned
✓ Impact camera capture pipeline (up to 240 FPS)	↺ Radar integration — current priority
✓ Full touch UI (profiles, bag, history, metrics, settings)	📋 Establish radar max accurate range
✓ Intrinsic camera calibration (checkerboard)	📋 Introduce camera; derive ball distance
✓ Ball-zone state machine	📋 Calibrate real geometry
✓ Development Mode (runs on laptop, simulated)	📋 Sensor fusion (camera spin + radar)
✓ Windows desktop staging build	📋 Spin measurement + ballistics

The **~5 ft** camera distance and **8 mm lens** are the starting hypothesis for step 3, not settled facts. Expect these to change once real radar range and camera pixel-size data are in hand.

Physical build (in progress)



CAD render of the current two-bay test tower — electronics bay (Pi 5 + power) on one side, angled radar mounts on the other. Camera mount not yet added.

A **two-bay tower** enclosure is being designed in CAD as a **radar + electronics test rig**: one bay for the Pi 5 and power (TalentCell + buck), one bay with angled mounts for the three radars. The **camera mount is still TBD** — its height and vertical position relative to the ball need to be determined for optimal capture, which depends on the lens FOV and the (not-yet-fixed) camera distance. Screen mounting and final packaging come in a later revision once all hardware is in hand. This is a **test rig first, product enclosure later**.

7. Key open question — spin detection

The biggest research unknown: **can spin be measured on an unmarked golf ball, or do we need specially marked balls?**

- The commercial Rapsodo MLM2 Pro **requires** RPT-dotted balls for spin — without them it reports "N/A." This suggests even Rapsodo couldn't reliably measure spin from unmarked balls at this price point.
- Options under evaluation: logo/number tracking, dimple-pattern optical flow, ML approaches, TaylorMade Pix / Callaway Triple Track patterned balls, RPT-style dotted balls, or DIY sticker markers.
- The answer drives a hard requirement: **how many pixels of ball diameter** we need on target, which in turn drives the camera distance and lens choice in step 3 above.

Currently, spin is **not yet implemented** — the camera captures the frames, but spin analysis (RPM, axis) isn't coded, and the UI shows placeholder values.

8. The short version

A Raspberry Pi 5 with a high-speed global-shutter camera (spin) and three 24 GHz Doppler radars (ball/club speed, launch angle, club path) measures a golf shot and shows the numbers on an 800×480 touchscreen. An optional sound trigger can catch the impact; a C++ backend fuses camera + radar data; Python drives the radars; Qt/QML runs the UI. Power comes from a 5 V/5 A source — a wall PD supply, a PD power bank, or a 12 V TalentCell battery through a buck converter for a portable tower.

The software foundation and full UI are built and working in simulation. The immediate focus is getting the radars operational, then fitting the camera geometry around the radars' real accurate range. The biggest open question is whether spin can be read from unmarked balls or whether marked balls are required.